ТОО ”Колледж Хекслет”

Лабораторная работа №1

Создание первого серверного приложения на Express.js

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_2. Создание первого серверного приложения на Express.js.

Цель: Научиться создавать Node.js-проект, устанавливать Express.js, запускать HTTP-сервер, создавать GET-маршруты и возвращать клиенту текстовые и JSON-ответы.

Условие: RO4_2/lesson1/ЛР-1.pdf

Требования задания

• использовать Express.js;

• использовать синтаксис ES Modules;

• запускаться командой npm start;

• работать на порту 3000;

• содержать шесть обязательных GET-маршрутов;

• возвращать текстовые ответы через res.send();

• возвращать объекты и массивы через res.json();

• содержать минимум пять товаров;

• содержать минимум три категории;

• запускаться без ошибок.

Ход выполнения

Создан ES Modules проект с Express и npm-командой start. Сервер возвращает текст через res.send(), объекты и массивы через res.json(). В каталоге пять товаров, в списке три категории.

Проект: RO4_2/lesson1

Запуск из каталога проекта: npm install, затем npm start. Адрес сервера: http://localhost:3000.

package.json

```json
{
  "name": "ro4_2-lesson1",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "start": "node app.js",
    "dev": "node --watch app.js"
  },
  "dependencies": {
    "express": "^5.1.0"
  }
}
```

Листинг 1. package.json

app.js

```js
import express from 'express';
import { pathToFileURL } from 'node:url';
const app = express();
let products = [
    {
        "id": 1,
        "name": "Keyboard",
        "price": 15000,
        "category": "Accessories",
        "inStock": true
    },
    {
        "id": 2,
        "name": "Mouse",
        "price": 8000,
        "category": "Accessories",
        "inStock": true
    },
    {
        "id": 3,
        "name": "Monitor",
        "price": 85000,
        "category": "Monitors",
        "inStock": false
    },
    {
        "id": 4,
        "name": "Laptop",
        "price": 420000,
        "category": "Computers",
        "inStock": true
    },
    {
        "id": 5,
        "name": "Webcam",
```

Листинг 2. app.js (часть 1)

app.js

```
        "price": 22000,
        "category": "Accessories",
        "inStock": false
    }
];
let nextId = 6;
const categories = ['Computers', 'Accessories', 'Monitors'];
app.get('/', (req, res) => res.send('Internet Shop API'));
app.get('/about', (req, res) => res.send('Учебный REST API магазина компьютерной
техники'));
app.get('/api/status', (req, res) => res.json({ status: 'ok', message: 'Server is
running' }));
app.get('/api/shop', (req, res) => res.json({ name: 'Kirill Tech Store', description:
'Учебный интернет-магазин', currency: 'KZT' }));
function getProducts(req, res) { res.json(products); }
app.get('/api/products', getProducts);
app.get('/api/categories', (req, res) => res.json(categories));
export default app;
if (process.argv[1] && import.meta.url === pathToFileURL(process.argv[1]).href) {
    const PORT = Number(process.env.PORT || 3000);
    app.listen(PORT, () => console.log(`Server: http://localhost:${PORT}`));
}
```

Листинг 3. app.js (часть 2)

Проверка результата

Проверено: 05.11.2026 13:44 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Обязательные endpoints

• HTTP-статусы и содержимое ответов

HTTP-запросы

GET / → 200

GET /about → 200

GET /api/status → 200

GET /api/shop → 200

GET /api/products → 200

GET /api/categories → 200

Примеры ответов

GET / → 200

"Internet Shop API"

GET /about → 200

"Учебный REST API магазина компьютерной техники"

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.